



Trabalho Prático

Computação Gráfica - Fase I

2025/2026

Grupo 50

Francisco Contente - A106846 Gustavo Braga - A107379 Lucas Pinto - A107288

Índice Geral

1. Introdução	1
2. Generator	2
2.1. Funcionamento	2
2.1.1. Estrutura do ficheiro .3d e características dos Triângulos	2
2.1.2. Utilização do programa Generator	3
2.2. Primitivas geométricas	5
2.2.1. Plano	5
2.2.2. Cubo	7
2.2.3. Esfera	9
2.2.3.1. Discretização da superfície	9
2.2.3.2. Definição dos Ângulos	9
2.2.3.4. Degeneração nos polos	11
2.2.4. Cone	11
2.2.4.1. Superfície Lateral	11
2.2.4.2. Triangulação da Superfície	12
2.2.4.3. Base Circular	14
2.2.5. Cilindro	15
2.2.5.1. Superfície Lateral	15
2.2.5.2. Bases Superior e Inferior	16
3. Engine	18
3.1. Arquitetura	18
3.2. Leitura e Parsing de Cenas (XML)	18
3.3. Gestão de modelos	20
3.4. Câmera	20
3.4.1. Movimentação da câmera	20
3.4.1.1. Interação com Inputs do Utilizador	20
3.4.1.2. Conversão para Coordenadas Cartesianas	21
3.5. Renderização	21
4. Resultados obtidos	22
4.1. Figuras geométricas geradas	22
4.2. Resultado dos testes	23
5. Conclusão	25
ANEXO: GUIA DE UTILIZAÇÃO E EXECUÇÃO	26
1. Compilação do Projeto	26
2. Geração de modelos (Generator)	26
3. Execução da engine	26

Índice de Figuras

Figura 1 - Representação dos vértices de uma célula [ABCD] no plano	6
Figura 2 - Representação dos vértices de uma célula [ABCD] da face superior do cubo	8
Figura 3 - Representação dos vértices de uma célula [ABCD] na esfera	10
Figura 4 - Representação dos vértices de uma célula [ABCD] no cone	13
Figura 5 - Representação de um triângulo [ABC] na base do cone	14
Figura 6 - Representação de um quadrilátero [ABCD] na superfície lateral de um cilindro	16
Figura 7 - Representação de um triângulo[ABC] na base superior de um cilindro	16
Figura 8 - Exemplo de configuração XML	19
Figura 9 - Plano	22
Figura 10 - Cubo	22
Figura 11 - Esfera	22
Figura 12 - Cone	22
Figura 13 - Cilindro	23
Figura 14 - Teste 1	23
Figura 15 - Teste 2	23
Figura 16 - Teste 3	24
Figura 17 - Teste 4	24
Figura 18 - Teste 5	24

1. Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Computação Gráfica (CG), do 3º ano, 2º semestre, do curso de Licenciatura em Engenharia Informática (LEI), no ano letivo 2025/2026. Este descreve o trabalho desenvolvido na Fase 1 do projeto desta unidade curricular.

Esta primeira fase consistiu no desenvolvimento de duas aplicações distintas: o Generator, responsável pela criação dos modelos geométricos através da geração dos respetivos vértices e armazenamento em ficheiros .3d, e a Engine, encarregue de ler um ficheiro de configuração em formato XML contendo as definições da janela e câmara e a referência aos modelos a carregar, procedendo posteriormente à sua representação gráfica.

2. Generator

2.1. Funcionamento

A primeira tarefa deste trabalho prático consistiu no desenvolvimento do Generator. Este é responsável por criar os modelos geométricos utilizados pela engine, produzindo os vértices que compõem os vários triângulos responsáveis pela renderização das primitivas geométricas.

Cada primitiva é gerada com base em parâmetros específicos definidos pelo utilizador, sendo o resultado final um ficheiro .3d, composto por todos os vértices necessários para que a Engine possa interpretar e construir graficamente a superfície correspondente.

2.1.1. Estrutura do ficheiro .3d e características dos Triângulos

Os ficheiros .3d gerados pelo Generator contêm todos os vértices necessários para representar as primitivas geométricas, permitindo à Engine construí-las corretamente.

A estrutura do ficheiro é a seguinte:

- Número total de vértices - a primeira linha do ficheiro indica o número total de pontos existentes.
- Lista de vértices - cada linha subsequente contém as coordenadas de um vértice, no formato `x y z`, separadas por espaços.

Cada conjunto de 3 vértices consecutivos e sequenciais define um triângulo da primitiva, ou seja, os vértices são interpretados pela Engine em grupos fixos de três, sem sobreposição entre triângulos. Esta organização linear simplifica a leitura do ficheiro e permite reconstruir a geometria de forma direta e eficiente.

Na geração de cada primitiva, os triângulos são construídos com os vértices em sentido anti-horário, garantindo que as faces visíveis estejam orientadas para o exterior. Esta

orientação define corretamente a direção das normais (vetores perpendiculares à superfície de cada triângulo e que indicam para que lado a face está apontada). A consistência na ordem dos vértices permite utilizar *face culling*, ignorando automaticamente os triângulos voltados para o interior da primitiva e evitando artefactos visuais. Esta convenção é aplicada a todas as primitivas, tanto planas como curvas.

Cada célula quadrada é decomposta em 2 triângulos, seguindo a mesma convenção anti-horária.

2.1.2. Utilização do programa Generator

O Generator é executado através da linha de comandos, recebendo como argumentos o tipo de primitiva a gerar, os parâmetros correspondentes e o nome do ficheiro de saída. De forma geral, a execução segue o seguinte padrão:

```
generator <primitiva> <parâmetros> <nome_ficheiro.3d>
```

Cada primitiva geométrica possui um conjunto específico de parâmetros, conforme descrito abaixo:

Plano

- size - comprimento do lado do plano;
- divisions - número de divisões ao longo de cada eixo.

Exemplo de execução:

```
generator plane 5 10 plane.3d
```

Cubo

- size - comprimento da aresta do cubo;
- divisions - número de subdivisões por face.

Exemplo de execução:

```
generator box 2 4 box.3d
```

Esfera

- radius - raio da esfera;
- slices - número de divisões verticais;
- stacks - número de divisões horizontais.

Exemplo de execução:

```
generator sphere 1 20 20 sphere.3d
```

Cone

- radius - raio da base;
- height - altura;
- slices - número de fatias ao redor do eixo;
- stacks - número de subdivisões ao longo da altura.

Exemplo de execução:

```
generator cone 1 2 20 10 cone.3d
```

Cilindro

- radius - raio da base;
- height - altura do cilindro;
- slices - número de fatias ao redor do eixo.

Exemplo de execução:

```
generator cylinder 1 2 20 cylinder.3d
```

As primitivas plano, cubo, esfera e cilindro são geradas centradas na origem do sistema de coordenadas, enquanto o cone possui a base assente no plano XZ.

Em todos os casos, o nome do ficheiro de saída pode ser definido livremente, permitindo organizar os modelos conforme necessário. O ficheiro gerado contém todos os vértices necessários para que a Engine construa corretamente a primitiva.

2.2. Primitivas geométricas

O Generator permite a criação das primitivas fundamentais especificadas no enunciado do projeto: plano, cubo, esfera e cone. Cada uma destas figuras é definida por parâmetros geométricos próprios e representada através de triângulos que descrevem a sua superfície.

Para além das primitivas obrigatórias, foi ainda implementado o cilindro como primitiva adicional. Esta inclui a geração da superfície lateral e das respetivas bases circulares, mantendo a mesma lógica de construção e organização dos triângulos utilizada nas restantes figuras.

Nos pontos seguintes é apresentada, de forma detalhada, a definição geométrica e o processo de geração de cada primitiva.

2.2.1. Plano

A primitiva plano constitui a base geométrica para a construção de superfícies bidimensionais no espaço tridimensional. Conforme descrito anteriormente, esta primitiva é definida pelo comprimento do lado e pelo número de subdivisões ao longo de cada eixo.

Para que o plano ficar centrado na origem (0,0,0), define-se o domínio espacial no intervalo:

$$I = \left[-\frac{size}{2}, \frac{size}{2} \right]$$

Este valor é guardado na variável *start*. Desta forma, o plano estende-se simetricamente ao longo dos eixos X e Z, permanecendo fixo no eixo Y (isto é, $y = 0$).

O espaçamento entre pontos consecutivos da grelha é calculado por:

$$step = \frac{size}{divisions}$$

Obtém-se assim uma superfície regular composta por $divisions \times divisions$ quadrados.

A geração geométrica é realizada através de dois ciclos aninhados, o primeiro sobre o eixo Z (variável i) e o segundo sobre o eixo X (variável j). Para cada célula da grelha são determinados quatro pontos no plano XZ:

$$\begin{aligned}x_1 &= start + j * step \\x_2 &= start + (j + 1) * step \\z_1 &= start + i * step \\z_2 &= start + (i + 1) * step\end{aligned}$$

Sendo os quatro vértices de um quadrado, representados na figura abaixo:

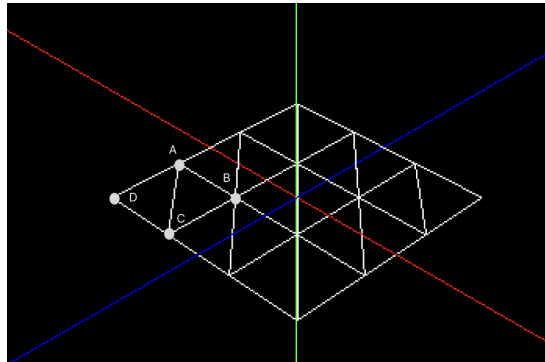


Figura 1 - Representação dos vértices de uma célula [ABCD] no plano

Em que cada quadrado é então decomposto em dois triângulos:

$$T_1 = (A, D, C) \quad T_2 = (A, C, B)$$

A ordem dos vértices é definida em sentido anti-horário quando observada do semi-eixo positivo de Y, garantindo que a normal da superfície aponta para cima e evitando problemas de *face-culling*.

Para permitir que o plano seja visível tanto a partir de cima como de baixo, foram igualmente gerados os triângulos correspondentes à face inferior. Para tal, adicionam-se dois novos triângulos por cada célula, com a ordem dos vértices invertida, garantindo que a normal fique orientada no sentido negativo do eixo Y. Desta forma, o plano passa a possuir dupla face, assegurando a sua visualização independentemente da posição da câmera.

Desta forma, é gerado um conjunto estruturado de vértices que descreve integralmente a geometria do plano, permitindo à engine gráfica reconstruir corretamente a superfície através da renderização dos triângulos definidos.

2.2.2. Cubo

A geração da primitiva cubo baseia-se na construção de seis planos ortogonais no espaço. Para garantir que o sólido geométrico seja centrado na origem (0,0,0), define-se o domínio de cada eixo no intervalo fechado:

$$I = \left[-\frac{size}{2}, \frac{size}{2} \right]$$

Por conveniência, define-se $half = \frac{size}{2}$, de forma a simplificar a notação ao longo da construção das faces.

Cada face é tratada como uma superfície subdividida numa grelha com tamanho $divisions \times divisions$ de células. O espaçamento entre vértices consecutivos é dado por:

$$step = \frac{size}{divisions}$$

A construção geométrica é executada através de dois ciclos aninhados que iteram sobre os índices $i, j \in [0, divisions - 1]$, definindo as coordenadas locais de cada célula. Estas coordenadas são calculados através do uso de 4 pontos:

- $a = -half + j \cdot step$
- $b = -half + (j + 1) \cdot step$
- $c = -half + i \cdot step$
- $d = -half + (i + 1) \cdot step$

Para cada face, uma coordenada permanece constante, enquanto as outras duas variam de acordo com os limites da célula:

Face	Coordenadas Fixas	Coordenadas Variáveis
------	-------------------	-----------------------

Topo	y = half	X,Z
Base	y = -half	X,Z
Frontal	z = half	X,Y
Posterior	z = -haf	X,Y
Direita	x = half	Y,Z
Esquerda	x = -half	Y,Z

Tomando como referência a célula [ABCD] representada na figura, a construção dos triângulos varia consoante a face para garantir que a normal aponte para fora:

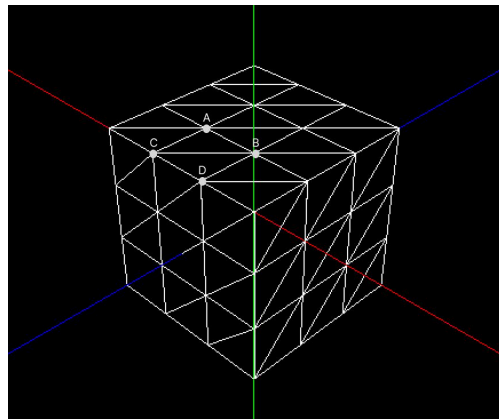


Figura 2 - Representação dos vértices de uma célula [ABCD] da face superior do cubo

Para que estes triângulos estejam voltados para o exterior do cubo os triângulos tem que estar ordenados seguindo um sentido anti-horário:

$$T_1 = (A, C, B) \quad T_2 = (C, D, B)$$

Caso esta célula tivesse na face inferior os triângulos estariam ordenados do seguinte modo:

$$T_1 = (A, B, C) \quad T_2 = (C, B, D)$$

2.2.3. Esfera

Para a geração da esfera foi necessário utilizar dois ângulos para o cálculo dos vértices: ϕ (latitude), que varia entre $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$, e θ (longitude) que varia entre $[0, 2\pi]$. Um ponto da esfera é dado pelas seguintes equações paramétricas:

$$\begin{aligned}x &= r \cos(\phi) \sin(\theta) \\y &= r \sin(\phi) \\z &= r \cos(\phi) \cos(\theta)\end{aligned}$$

2.2.3.1. Discretização da superfície

A esfera é dividida em *slices* (fatias verticais) e *stacks* (fatias horizontais). O cruzamento destas divisões forma uma grelha de quadriláteros sobre a superfície esférica. Os incrementos angulares em cada direção são calculados da seguinte forma:

$$sliceStep = \frac{2\pi}{slices}$$

Este valor corresponde ao incremento angular na direção horizontal (longitude), sendo responsável pela distribuição dos pontos ao longo do eixo horizontal da esfera.

$$stackStep = \frac{\pi}{stacks}$$

Este valor representa o incremento angular na direção vertical (latitude), determinando os diferentes níveis onde os pontos são posicionados ao longo da esfera.

2.2.3.2 Definição dos Ângulos

Para cada célula da malha paramétrica, consideram-se dois ângulos consecutivos de latitude e dois de longitude.

Onde ϕ_1 e ϕ_2 são calculados a partir do polo norte ($\frac{\pi}{2}$) descendo progressivamente:

$$\phi_1 = \frac{\pi}{2} - i \cdot stackStep \quad \phi_2 = \frac{\pi}{2} - (i + 1) \cdot stackStep$$

E θ_1 e θ_2 são calculados a partir da origem, avançando progressivamente ao longo dos *slices*:

$$\theta_1 = j. \text{ sliceStep} \quad \theta_2 = (j + 1). \text{ sliceStep}$$

Onde $i \in [0, \text{stacks} - 1]$ percorre as divisões verticais e $j \in [0, \text{slices} - 1]$ percorre as divisões horizontais.

2.2.3.3. Triangulação da Malha

Por fim, procedemos à triangulação da superfície da esfera. O cruzamento das divisões verticais e horizontais forma uma malha paramétrica composta por quadriláteros sobre a superfície esférica. Cada célula é identificada por 4 vértices.

Vértice	ϕ	θ
A	ϕ_1	θ_1
B	ϕ_1	θ_2
C	ϕ_2	θ_2
D	ϕ_2	θ_1

No exemplo abaixo segue uma possível representação da célula com vértices [ABCD]

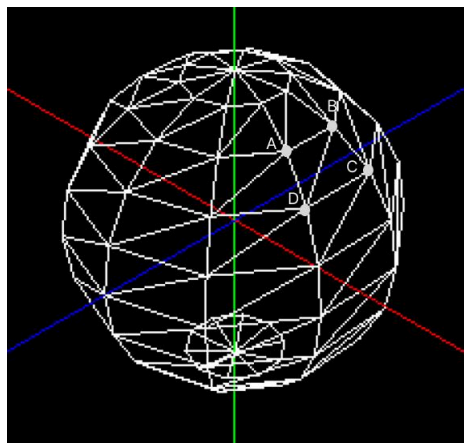


Figura 3 - Representação dos vértices de uma célula [ABCD] na esfera

Os triângulos daí provenientes são:

$$T_1 = (A, D, B) \quad T_2 = (B, D, C)$$

2.2.3.4. Degeneração nos polos

Na primeira e última iteração das *stacks* (polo norte e polo sul), ocorre um fenómeno de degeneração de triângulos:

- No polo norte ($i = 0$), a latitude $\phi_1 = \frac{\pi}{2}$. Como $\cos(\frac{\pi}{2}) = 0$, os vértices A e B correspondem à mesma coordenada espacial $(0, r, 0)$. Consequentemente, o triângulo T_1 torna-se degenerado, isto é, a área é zero, enquanto que o triângulo T_2 assume a forma de triângulo que fecha o topo da esfera.
- No polo sul ($i = \text{stacks}-1$), a latitude $\phi_2 = -\frac{\pi}{2}$. Como $\cos(-\frac{\pi}{2}) = 0$, os vértices C e D reduzem-se na mesma coordenada espacial $(0, -r, 0)$. Neste caso, é o triângulo T_2 que degenera, e o triângulo T_1 que constitui um triângulo válido.

Para corrigir estes casos excepcionais, na primeira iteração é ignorada a construção do triângulo T_1 , e na última iteração é ignorada a construção do triângulo T_2 . Em todas as outras camadas intermédias, ambos os triângulos são gerados para formar a malha retangular da superfície.

2.2.4. Cone

A geração do cone é decomposta em duas entidades geométricas distintas: a superfície lateral cónica e a base circular. O modelo é definido pelo raio da base R , a altura H , e os parâmetros de discretização *slices* e *stacks*.

2.2.4.1. Superfície Lateral

O raio do cone varia linearmente ao longo do eixo dos y até à sua altura. Para cada nível $i \in [0, \text{stacks}-1]$, o raio local r_i e a altura y_i correspondente são calculados por:

$$r_i = R \times \left(1 - \frac{i}{\text{stacks}}\right) \quad ; \quad y_i = i \times \left(\frac{H}{\text{stacks}}\right)$$

Esta formulação garante que na base ($i=0$), o raio seja R , e no ápice ($i=stacks$), o raio seja nulo, convergindo todos os vértices para o ponto $(0,H,0)$.

A altura é subdividida em stacks definidos por:

$$stackStep = \frac{H}{stacks}$$

Simultaneamente cada fatia ao redor do eixo (slice) tem incremento angular:

$$sliceStep = \frac{2\pi}{slices}$$

Para cada célula da malha, os ângulos consecutivos são calculados como:

$$\theta_1 = j \cdot sliceStep \quad \theta_2 = (j + 1) \cdot sliceStep$$

E as alturas dos dois níveis adjacentes como:

$$y_1 = i \cdot stackStep \quad y_2 = (i + 1) \cdot stackStep$$

As coordenadas espaciais (x,y,z) de cada vértice são obtidas através da conversão de coordenadas cilíndricas para cartesianas:

$$x = r_i \sin(\theta_j)$$

$$y = i \cdot y_i$$

$$z = r_i \cos(\theta_j)$$

onde $j \in [0, slices]$ representa o índice da subdivisão radial.

2.2.4.2. Triangulação da Superfície

Para cada célula da malha definem-se quatro vértices com base nos raios e ângulos consecutivos:

Vértice	Raio	θ	Altura
A	r_2	θ_1	y_2
B	r_2	θ_2	y_2
C	r_1	θ_2	y_1
D	r_1	θ_1	y_1

A malha é construída interligando vértices de níveis adjacentes (i e $i+1$). Cada secção da superfície é aproximada por dois triângulos. Os vértices acima definidos podem ser descritos como se apresenta na seguinte imagem:

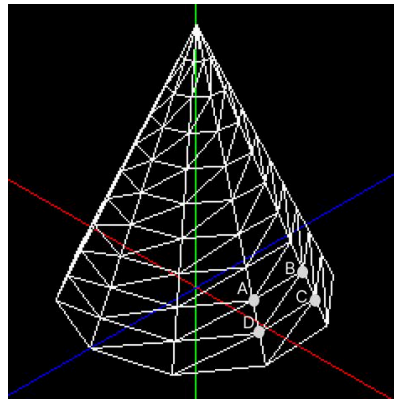


Figura 4 - Representação dos vértices de uma célula [ABCD] no cone

Os triângulos são construídos seguindo a ordem anti-horária para garantir que as normais apontem para o exterior:

$$T_1 = (A, D, B) \quad T_2 = (B, D, C)$$

À semelhança do fenómeno observado na esfera, na última iteração ($i=\text{stacks}-1$), r_{stacks} atinge o valor zero. Nesta geometria, os vértices A e B coincidem na mesma coordenada espacial (0,H,0), o que resultaria na criação de um triângulo degenerado. Para corrigir esta anomalia, o código foi estruturado para que, nesta última camada, seja construído apenas um triângulo funcional. Uma vez que os pontos A e B são coincidentes, utiliza-se indiferentemente um deles como vértice comum, ligando-o aos dois pontos da base inferior.

2.2.4.3. Base Circular

A base circular, situada no plano XZ com $y=0$, é subdividida em *slices*, triângulos que convergem para o centro da base na origem (0,0,0). Para cada fatia k , calculam-se dois ângulos consecutivos:

$$\theta_1 = k \cdot sliceStep \quad \theta_2 = (k + 1) \cdot sliceStep$$

Cada triângulo da base é formado pelos vértices:

Vértices	Coordenadas
Centro	(0,0,0)
A	$(R \cdot \sin(\theta_1), 0, R \cdot \cos(\theta_1))$
B	$(R \cdot \sin(\theta_2), 0, R \cdot \cos(\theta_2))$

Juntando esses dois pontos ao centro da base, forma-se um triângulo que representa uma fatia da base, como mostra a figura abaixo:

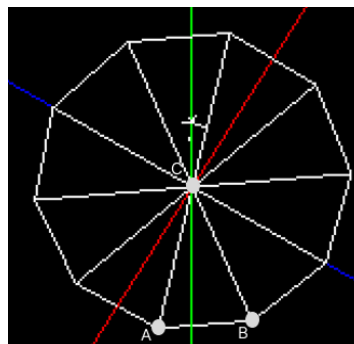


Figura 5 - Representação de um triângulo [ABC] na base do cone

A ordem dos vértices deve ser:

$$T_1 = (C, B, A)$$

Repetindo o procedimento para todas as *slices*, a base completa é representada por uma sequência de triângulos que cobrem todo o disco.

2.2.5. Cilindro

A geração do cilindro é decomposta em três entidades geométricas distintas: a base superior, a base inferior e a superfície lateral. O modelo é definido pelo raio R , a altura H , e o número de *slices*.

Para garantir que o cilindro fique centrado na origem do referencial cartesiano, define-se $half = \frac{H}{2}$, fazendo com que a base superior se situe em $y=+half$ e a base inferior em $y=-half$.

A discretização da circunferência é feita através do parâmetro *slices*, definindo o incremento angular:

$$sliceStep = \frac{2\pi}{slices}$$

Para cada divisão $j \in [0, slices-1]$, consideram-se dois ângulos consecutivos

$$\theta_1 = j \cdot sliceStep \quad \theta_2 = (j + 1) \cdot sliceStep$$

E os respectivos pontos na circunferência através da conversão de coordenadas polares para cartesianas:

$$\begin{aligned} x &= R \sin(\theta) \\ y &\in [-half, half] \\ z &= R \cos(\theta) \end{aligned}$$

2.2.5.1. Superfície Lateral

A superfície lateral é formada pela ligação entre os perímetros das duas bases. Cada *slice* define um quadrilátero que conecta dois pontos consecutivos da base superior com os correspondentes da base inferior.

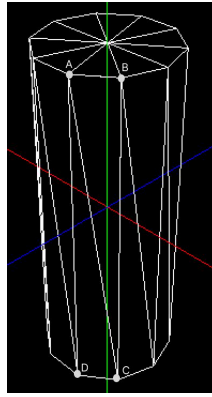


Figura 6 - Representação de um quadrilátero [ABCD] na superfície lateral de um cilindro

Considerando os pontos representados na figura acima, a triangulação é realizada da seguinte forma:

$$T_1 = (A, D, C) \quad T_2 = (B, A, C)$$

2.2.5.2. Bases Superior e Inferior

As bases são constituídas por um conjunto de triângulos que convergem para o centro de cada base. A ordem dos vértices é crítica para garantir que as normais apontem para fora do sólido:

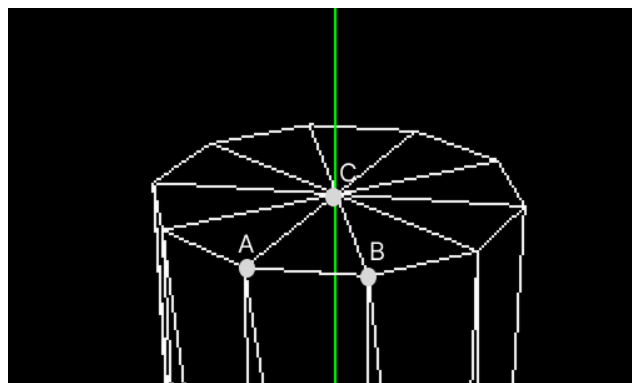


Figura 7 - Representação de um triângulo[ABC] na base superior de um cilindro

Na base superior os triângulos teriam a seguinte orientação:

$$T_1 = (C, A, B)$$

Na base inferior os triângulos seriam construídos no sentido contrário:

$$T_1 = (C, B, A)$$

Após a construção das bases inferior e superior e da superfície lateral, a representação completa de um cilindro fica concluída.

3. Engine

3.1. Arquitetura

A Engine utiliza o OpenGL 2.1 para a renderização, isolando as responsabilidades de parsing, geração de geometria e visualização.

Para a gestão da janela de visualização e do contexto gráfico, utilizamos a biblioteca GLUT, que permite controlar o ciclo de eventos principal e as funções de callback para processar a imagem e redimensionar a janela.

A interatividade com o utilizador é também assegurada pela GLUT, que processa o input do teclado. Isto permite que a posição e orientação da câmara sejam alteradas durante a execução do programa de acordo com os comandos do utilizador.

Estruturalmente, o código está dividido em componentes com responsabilidades distintas. O ficheiro *parser.cpp* é responsável pela interpretação de ficheiros XML e leitura dos modelos geométricos.

O ficheiro *engine.cpp* foca-se na configuração da visualização e nos algoritmos de desenho. Esta organização permite que a lógica de processamento de dados esteja separada da lógica de renderização.

A estrutura é suportada por um conjunto de definições localizadas no cabeçalho *structs.h*. Este ficheiro uniformiza a representação de pontos, modelos e parâmetros da câmara.

3.2. Leitura e Parsing de Cenas (XML)

O módulo parser é responsável pelo processamento e leitura do ficheiro XML. Para a análise do ficheiro, foi utilizada a biblioteca *TinyXML*. A seguir vemos um exemplo de uma cena em XML:

```

<world>
  <window width="800" height="600" />
  <camera>
    <position x="5" y="5" z="5" />
    <lookAt x="0" y="0" z="0" />
    <up x="0" y="1" z="0" />
    <projection fov="45" near="1" far="1000" />
  </camera>
  <group>
    <models>
      <model file="sphere.3d" />
    </models>
  </group>
</world>

```

Figura 8 - Exemplo de configuração XML

O processo inicia-se com a procura do elemento *world* que funciona como o contentor da cena. A partir deste nó principal, o sistema extrai as configurações globais da janela de visualização. O parser consulta o elemento *window* para obter os atributos de largura (*width*) e altura (*height*), que são posteriormente utilizados para definir as dimensões da janela na GLUT.

A seguir trata-se dos elementos da câmara que é responsável pela observação da cena. O código recupera as coordenadas espaciais para a posição do observador, o ponto de foco definido em *lookAt* e o vetor de orientação vertical *up*. Estes dados são fundamentais para alimentar a função de visualização do OpenGL.

Os parâmetros da câmara também incluem o campo projeção, que está no subelemento *projection*. São guardados os valores do campo de visão (*fov*), que representa o ângulo de abertura da câmara, e as distâncias dos planos de recorte próximo (*near*), indicando a distância mínima que um objeto deve estar da câmara, e distante (*far*), que define o limite máximo de visibilidade da câmara, garantindo que a perspetiva é renderizada conforme definido no ficheiro de teste.

A definição dos objetos a apresentar na cena é processada através da análise do elemento *group*. Dentro deste contentor, o *parser* identifica a *tag models* e percorre todos os elementos *model* presentes para recolher o caminho de cada ficheiro .3d.

Este fluxo garante que todos os vértices e triângulos gerados previamente pelo Generator sejam carregados para a memória da aplicação antes do início do ciclo de desenho.

3.3. Gestão de modelos

Após o programa fazer a leitura e processamento do ficheiro XML inicia-se o processamento dos dados. Para cada ficheiro .3d a função *load3DFile* é chamada, extraíndo as coordenadas dos vértices e armazenando numa estrutura do tipo *Ponto* que posteriormente é inserida numa estrutura *Model* que contém o conjunto de vértices que compõem o modelo.

O programa centraliza os modelos em uma variável global *sceneModels*.

Esta organização permite que a aplicação suporte cenas com múltiplos modelos distintos, carregando-os sequencialmente conforme indicado no ficheiro XML.

3.4. Câmera

A câmara é representada por uma variável global do tipo *Câmara*. A posição da câmara no espaço é definida pelo vetor *position*, que estabelece as coordenadas cartesianas da posição da câmara. O vetor *lookAt* aponta para um ponto específico no espaço, enquanto o vetor *up* define a inclinação vertical da lente.

Após os dados da câmara serem carregados é chamada a função *syncCamera*. Esta função converte a posição cartesiana inicial em coordenadas esféricas, calculando o raio, o ângulo horizontal (*alpha*) e a elevação (*beta*).

O sistema de visualização utiliza também as definições de projeção extraídas do *parser* para configurar a perspetiva.

3.4.1. Movimentação da câmara

A navegação é baseada em um sistema de coordenadas esféricas, permitindo que a câmara orbite em torno de um ponto de forma fluida. O estado da câmara é definido por três variáveis principais: *Alpha*, *Beta*, *Radius*.

3.4.1.1. Interação com Inputs do Utilizador

O controlo é efetuado através das setas do teclado e das teclas de zoom, alterando os seguintes parâmetros:

- **Setas Esquerda/Direita (Horizontal):** Alteram o ângulo *Alpha*. Este parâmetro controla a rotação horizontal (azimute), permitindo girar a câmara "em volta" dos eixos Y.

- **Setas Cima/Baixo (Vertical):** Alteram o ângulo **Beta**. Este parâmetro controla a inclinação vertical (elevação).
 - *Nota:* O valor de Beta está limitado entre -1.5 e 1.5 radianos para evitar a inversão da câmara ao passar pelos polos.
- **Teclas 'W' e 'S' (Zoom):** Alteram o **Radius** r . Diminuir o raio aproxima a câmara da origem (Zoom In), enquanto aumentá-lo afasta a câmara (Zoom Out).

3.4.1.2. Conversão para Coordenadas Cartesianas

Como o OpenGL utiliza o sistema de coordenadas cartesianas (x , y , z) para a função *gluLookAt*, o programa converte os inputs esféricos em tempo real utilizando as seguintes fórmulas trigonométricas:

$$x = r \cdot \cos(\beta) \cdot \sin(\alpha)$$

$$y = r \sin(\beta)$$

$$z = r \cdot \cos(\beta) \cdot \cos(\alpha)$$

3.5. Renderização

Para a renderização utilizamos a biblioteca GLUT que é responsável por coordenar a atualização da imagem e os *inputs* do utilizador.

O processo de renderização da cena está na função *renderScene*, que começa por limpar os *buffers* para garantir que os restos das imagens anteriores não apareçam no ecrã. Depois, a câmara é configurada através do comando *gluLookAt*. A *renderScene* invoca a *drawSceneFromEngine*, que é a função encarregue de percorrer a lista global de modelos carregados na aplicação, que percorre a lista global de modelos carregados e, para cada um, inicia um bloco *glBegin(GL_TRIANGLES)* onde envia sequencialmente as coordenadas de cada vértice para a placa gráfica.

A interatividade da câmara permite que o utilizador explore os modelos a partir de qualquer ângulo em tempo real. Este movimento é processado através de funções de *callback* da GLUT que capturam as teclas premidas e atualizam os parâmetros esféricos da câmara.

Como mencionado na secção 3.4.1. o sistema utiliza um modelo de navegação orbital, onde a câmara se move sobre a superfície de uma esfera invisível centrada na origem da cena.

Sempre que ocorre uma alteração nestes valores, a Engine recalcula automaticamente as coordenadas cartesianas da posição da câmera.

Após a atualização dos dados, é invocada a função *glutPostRedisplay*, que sinaliza à Engine a necessidade de redesenhar a cena com a nova perspectiva. Este método assegura que o movimento pareça fluido e responsivo ao *input* do utilizador, sem interromper o fluxo de renderização.

A renderização termina com a função *glutSwapBuffers*, que troca o *buffer* de desenho pelo *buffer* de exibição. Utilizamos esta função para que o utilizador veja somente a imagem final da renderização.

4. Resultados obtidos

4.1. Figuras geométricas geradas

Os seguintes modelos foram criados utilizando o generator e a engine:

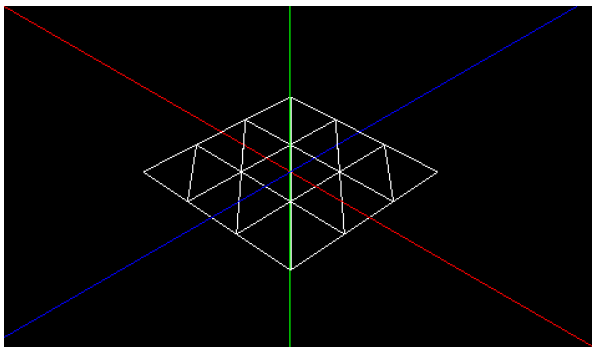


Figura 9 - Plano

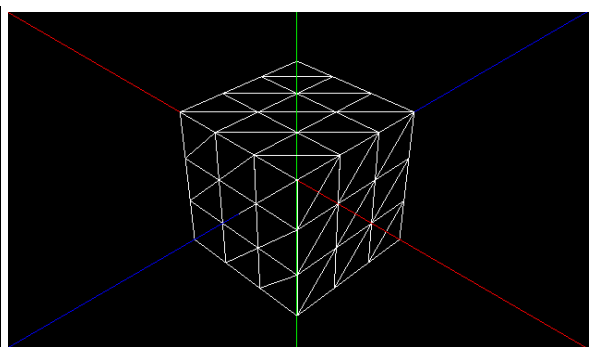


Figura 10 - Cubo

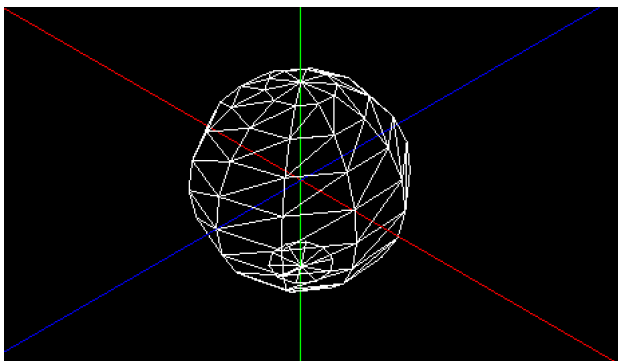


Figura 11 - Esfera

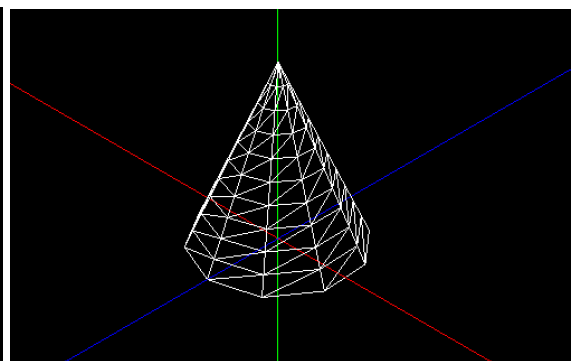


Figura 12 - Cone

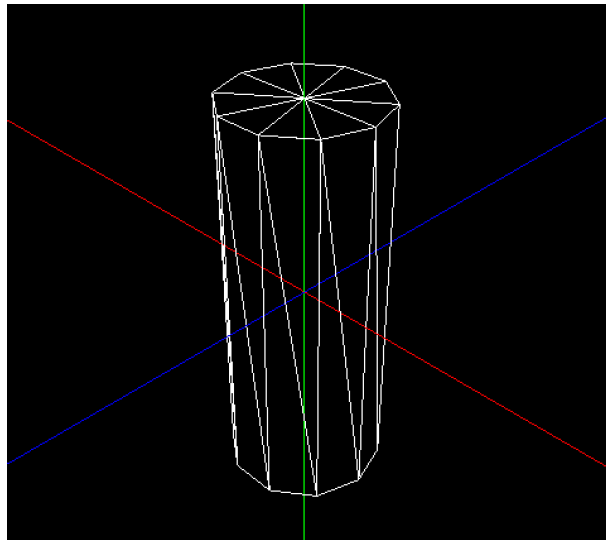


Figura 13 - Cilindro

4.2. Resultado dos testes

A seguir são apresentadas imagens da renderização dos testes fornecidos, mostrando que a engine foi capaz de renderizar todos os modelos com sucesso.

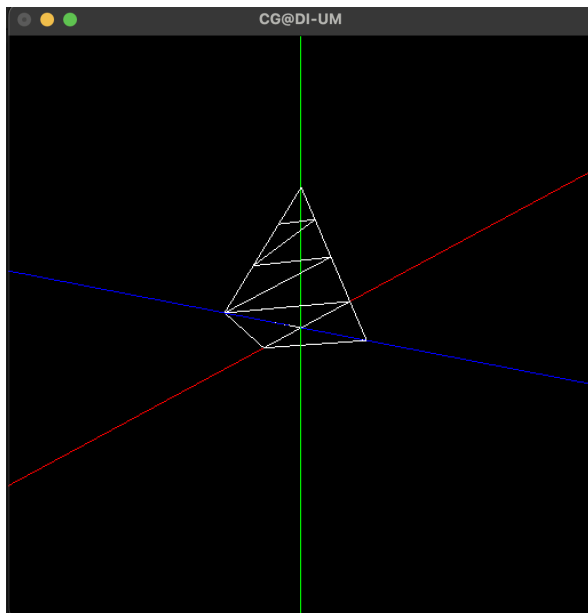


Figura 14 - Teste 1

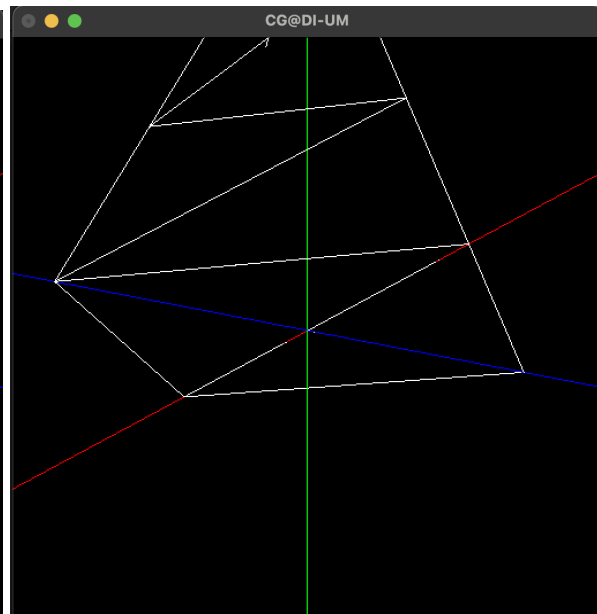


Figura 15 - Teste 2

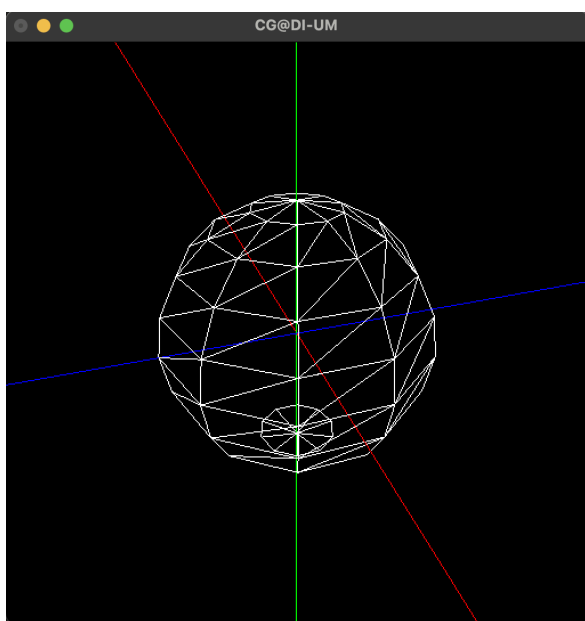


Figura 16 - Teste 3

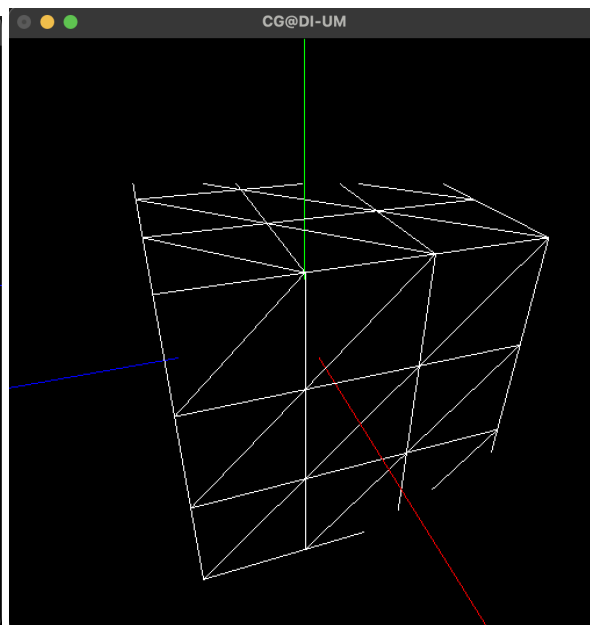


Figura 17 - Teste 4

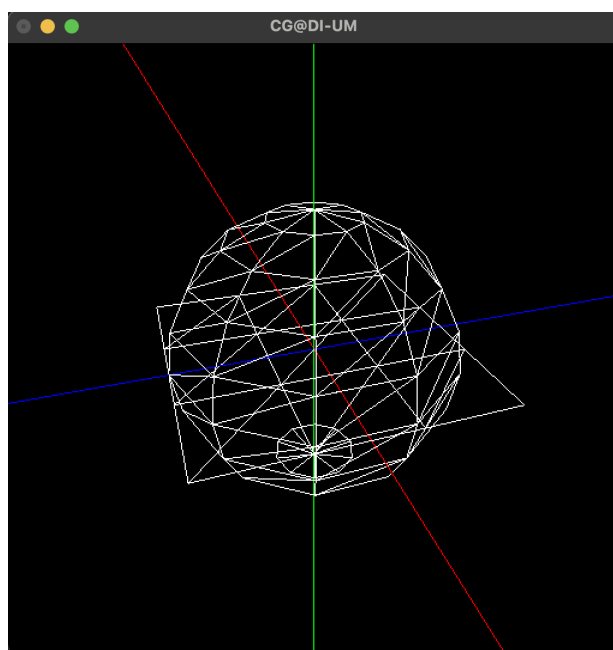


Figura 18 - Teste 5

5. Conclusão

A primeira fase do projeto foi concluída com sucesso, tendo sido alcançados todos os objetivos previamente definidos. A arquitetura implementada permitiu separar de forma clara a geração de geometria da sua visualização, garantindo um fluxo de trabalho eficiente entre o Generator e a Engine.

Além das primitivas bases (plano, caixa, esfera e cone), foi também implementada a geração do cilindro. A engine demonstrou ser capaz de processar estes ficheiros de forma robusta, mantendo uma performance estável mesmo com níveis de subdivisão elevados, validando assim a eficácia do *parser* e da lógica de renderização desenvolvida.

ANEXO: GUIA DE UTILIZAÇÃO E EXECUÇÃO

1. Compilação do Projeto

Para compilar o módulo generator e engine utilizamos os seguintes comandos:

1. **Criar pasta de compilação:** `mkdir build && cd build`
2. **Configurar o CMake:** `cmake ..`
3. **Compilar:** `make`

2. Geração de modelos (Generator)

Sintaxe do generator:

```
./generator <forma> [parâmetros] <ficheiro_saida.3d>
```

Exemplos de comandos:

- **Plano:** `./generator plane 2 10 plane.3d`
- **Caixa:** `./generator box 2 10 box.3d`
- **Esfera:** `./generator sphere 1 20 20 sphere.3d`
- **Cone:** `./generator cone 1 2 20 20 cone.3d`
- **Cilindro:** `./generator cylinder 1 2 20 cylinder.3d`

3. Execução da engine

Pode carregar qualquer ficheiro XML presente na pasta de cenas (copiada automaticamente para o diretório de execução pelo CMake):

- `./engine scenes/primitives.xml`
- `./engine scenes/stress_test.xml`
- `./engine scenes/curved_surfaces.xml`

Se o executável for chamado sem argumentos, será utilizado por padrão o ficheiro *test.xml* no momento da execução:

- `./engine`